

综合题

三种调度算法

题型1 计算平均周转时间和平均带权周转时间

有4个进程A、B、C、D，它们的到达时间、预计运行时间以及优先级数值，如表所示。

(1) 请计算采用短进程优先调度算法的平均周转时间和平均带权周转时间。

(2) 请计算采用抢占式优先权调度算法的平均周转时间和平均带权周转时间。

(注：精确到小数点后2位；优先级数值越小，表示优先级越高)

进程名	到达时间	预计运行时间	优先数
A	0	34	3
B	1	7	1
C	2	15	2
D	3	4	4

(1) 短进程优先调度算法

分析过程如下：

第0秒，只有A到达，执行A到第34秒

第34秒，BCD已经到达，从BCD中选择运行时间最短的D，执行D到38秒

第38秒，从BC中选择B，执行B到45秒，

最后，选择C执行C到60秒

所以调度顺序是A-D-B-C，完成时间分别是34-38-45-60

进程名	到达时间	完成时间	周转时间	运行时间	带权周转时间
A	0	34	34	34	1
B	1	45	44	7	6.29
C	2	60	58	15	3.87
D	3	38	35	4	8.75

平均周转时间 = $(34+44+58+35) / 4 = 42.75$

平均带权周转时间 = $(34/34+44/7+58/15+35/4) / 4=4.98$

(2) 抢占式优先权调度算法

分析过程：因为是抢占式调度，而且优先级越小就越有效

第0秒只有A到达，执行A到第1秒

第1秒B到达，优先级B比A高，B抢占A，A还剩余运行33秒

因为B优先级最高，无进程能抢占它，所以B运行完成到第8秒

第8秒，ACD都已到达，选择优先级最高的C，执行C到第23秒

第23秒，选择优先级最高的A，这时A只需要运行33秒，执行到第56秒

第56秒，选择D执行完成到第60秒

进程名	到达时间	完成时间	周转时间	运行时间	带权周转时间
A	0	56	56	34	1.65
B	1	8	7	7	1
C	2	23	21	15	1.4
D	3	60	57	4	14.25

平均周转时间 = $(56+7+21+57) / 4 = 35.25$

平均带权周转时间 = $(56/34+7/7+21/15+57/4) / 4=4.57$

先来先服务调度算法FCFS					
进程名	进入系统时间	开始运行时间	服务时间	等待时间	周转时间
p1	0	0	24	0	24
p2	1	24	3	23	26
p3	2	27	3	25	28

系统平均周转时间 $T = 1/n \times (T_1 + T_2 + T_3 + \dots + T_n) = 1/3 \times (24 + 26 + 28) = 26$

带权平均周转时间 $W = 1/n \times (T_1/T_{1s} + T_2/T_{2s} + T_3/T_{3s} + \dots + T_n/T_{ns}) = 1/3 \times (24/24 + 26/3 + 28/3) = 6.33$

短进程优先调度算法SPF					
进程名	进入系统时间	开始运行时间	服务时间	等待时间	周转时间
p1	2	6	24	4	28
p2	1	3	3	2	5
p3	0	0	3	0	3

系统平均周转时间 $T = 1/n \times (T_1 + T_2 + T_3 + \dots + T_n) = 1/3 \times (28 + 5 + 3) = 12$

带权平均周转时间 $W = 1/n \times (T_1/T_{1s} + T_2/T_{2s} + T_3/T_{3s} + \dots + T_n/T_{ns}) = 1/3 \times (28/24 + 5/3 + 3/3) = 1.28$

三种调度算法（扩展 - 可能不考）

多级队列 调度算法

系统中按进程类型设置多个队列，进程创建成功后插入某个队列

队列之间可采取固定优先级，或时间片划分

固定优先级：高优先级空时低优先级进程才能被调度

时间片划分：如三个队列分配时间50%、40%、10%

各队列可采用不同的调度策略，如：

- 系统进程队列采用优先级调度
- 交互式队列采用时间片轮转调度
- 批处理队列采用先来先服务调度

多级队列调度

多级反馈队列调度算法

例题：各进程到达就绪队列的时间、需要的运行时间如下表所示。使用多级反馈队列调度算法，分析进程运行的过程。

进程	到达时间	运行时间
P1	0	8
P2	1	4
P3	5	1

分析

设置多级就绪队列，各级队列**优先级从高到低**，**时间片从小到大**

新进程到达时**先进入第1级队列**，按**FCFS原则**排队等待被分配时间片。

若用完时间片进程还未结束，则进程**进入下一级队列队尾**；

如果此时**已经在最下级的队列**，则**重新放回最下级队列队尾**。

只有**第k级队列为空时**，才会为**k+1级队头**的进程**分配时间片**。

非抢占式

多级反馈队列 - 非抢占式

表格横向为时间轴，纵向为(i/q)轴，i表示队列，q表示时间片；

表格元素表示各就绪队列进程情况，A3表示A进程还需运行3个单位

抢占式

银行家算法

题型1 画资源分配状态表，找出安全序列，判断安全状态

设系统中有三种类型的资源A、B、C，资源数量分别为15、7、18，系统有五个进程P1、P2、P3、P4、P5，其最大资源需求量分别为(5,4,9)、(4,3,5)、(3,0,5)、(5,2,5)、(4,2,4)。在T0时刻，系统为各进程已经分配的资源数量分别为(2,1,2)、(3,0,2)、(3,0,4)、(2,0,4)、(3,1,4)。若系统采用银行家算法实施死锁避免策略，则请回答：

- (1) 列表画出T0时刻的资源分配状态表，在表中显示进程还需要的资源数量和系统可用的资源数量。
- (2) T0时刻是否为安全状态？若是，请给出安全序列。
- (3) 在T0时刻若进程P1请求资源(3,0,3)，是否能实施资源分配？为什么？
- (4) 在T0时刻若进程P4请求资源(2,0,1)，则是否能实施资源分配？为什么？

答案

(1)

进程	最大资源需求量 (A B C)	已分配资源数量 (A B C)	需要资源数量 (A B C)	可用资源数 (A B C)
P1	5 4 9	2 1 2	3 3 7	2 5 2
P2	4 3 5	3 0 2	1 3 3	
P3	3 0 5	3 0 4	0 0 1	
P4	5 2 5	2 0 4	3 2 1	
P5	4 2 4	3 1 4	1 1 0	

(2) 是安全状态，安全序列为：P3 - P2 - P4 - P5 - P1

(3) 不能。原因是可用资源不够

(4) 能分配，存在安全序列为：P3-P2-P4-P5-P1

分析

(2) 安全序列不唯一，P3或P5先执行，P1在最后

(3) 可用资源只有2个A，P1请求3个A，不能满足，所以不能分配

(4) 请求资源(2,0,1)不超过可用资源(2,5,2)，每个种类都是小于等于，所以模拟分配，得到分配后的资源状态表：

进程	最大资源容量 (A B C)	已分配资源数量 (A B C)	需要资源数量 (A B C)	可用资源数 (A B C)
P1	5 4 9	2 1 2	3 3 7	0 5 1
P2	4 3 5	3 0 2	1 3 3	
P3	3 0 5	3 0 4	0 0 1	
P4	5 2 5	4 0 5	1 2 0	
P5	4 2 4	3 1 4	1 1 0	

新的可用资源数 = 可用资源 - 请求资源 = (2,5,2) - (2,0,1) = (0,5,1)

P4新的已分配资源数 = P4原已分配 + 本次分配 = (2,0,4) + (2,0,1) = (4,0,5)

P4新的需求资源数 = P4原需求资源 - 本次分配 = (3,2,1) - (2,0,1) = (1,2,0)

第三章 进程调度与死锁

3.1 进程调度的功能与时机

功能

进程调度的功能由操作系统的进程调度程序完成
按照某种策略和算法从就绪态进程中为当前空闲的 CPU 选择在其上运行的新进程。

时机

- A. 进程正常结束；
- B. 进程阻塞；
- C. 有更高优先级进程到来；
- D. 时间片用完；
- E. 进程异常结束。

补充：一个进程被中断后，什么时间能再占用处理器是与进程调度的策略有关。

3.2 进程调度算法

选择调度算法的准则

好算法的标准（助记：周转响应与截止，系统处理利用率）

- 1) 周转时间短：作业从提交给系统开始，到作业完成，花费时间短
- 2) 响应时间快：从用户提交作业开始，到系统开始响应，花费时间短
- 3) 截止时间的保证：保证作业在“开始截止时间”前开始，在“完成截止时间”前完成
- 4) 系统吞吐量高：系统在单位时间内完成的作业量多
- 5) 处理机利用率好：CPU的利用率尽可能高

计算公式

周转时间：外存等待时间 + 就绪队列等待时间 + 执行时间 + 等待I/O操作完成

平均周转时间： $T = \frac{1}{n} \times (T_1 + T_2 + \dots + T_n)$

带权平均周转时间： $W = \frac{1}{n} \times (\frac{T_1}{T_{1s}} + \frac{T_2}{T_{2s}} + \dots + \frac{T_n}{T_{ns}})$ 服务时间Ts：一个作业在CPU上执行的总时间

的总时间

进程调度算法

进程调度算法

先来先服务调度算法 短进程优先调度算法	优先权调度算法 时间片轮转调度算法	多级队列调度算法 多级反馈队列调度算法
------------------------	----------------------	------------------------

一、先来先服务调度算法（FCFS）

含义：从就绪队列的队首选择最先到达就绪队列的进程，为该进程分配CPU。

周转时间计算

进程名	进入系统时间	开始运行时间	服务时间	等待时间	周转时间
p1	0	0	24	0	24
p2	1	24	3	23	26
p3	2	27	3	25	28

系统平均周转时间 $T = 1/n \times (T_1 + T_2 + \dots + T_n) = 1/3 \times (24 + 26 + 28) = 26$

带权 $W = 1/n \times (T_1/T_{1s} + \dots + T_n/T_{ns}) = 1/3 \times (24/24 + 26/3 + 28/3) = 6.33$

二、短进程优先调度算法（SPF）

含义：从就绪队列中选择估计运行时间最短的进程，为该进程分配CPU。

周转时间计算

进程名	进入系统时间	开始运行时间	服务时间	等待时间	周转时间
p1	2	6	24	4	28
p2	1	3	3	2	5
p3	0	0	3	0	3

系统平均周转时间 $T = 1/3 \times (28 + 5 + 3) = 12$
带权平均周转时间 $W = 1/3 \times (28/24 + 5/3 + 3/3) = 1.28$

优点：与FCFS算法相比，短进程优先算法能有效降低进程的平均等待时间，提高系统吞吐量

缺点：对长进程不利；

不能保证紧迫进程的处理；

进程长短由用户估计，不一定准确。

三、优先权调度算法

含义：该算法中，系统将CPU分配给就绪队列中优先权最高的进程

类型：抢占式：运行期间，有更高优先权的进程到来，就可以抢占CPU

非抢占式：运行期间，有更高优先权的进程到来，也不能剥夺CPU

优先权类型：

静态优先权：创建时确定，运行期间保持不变

动态优先权：创建时确定，随着进程推进或等待时间增加而改变

存在的问题：无穷阻塞（饥饿问题）

解决的方案：增加等待时间很长的进程的优先权（即，采用**老化技术**）

老化技术是一种动态调整进程优先级的方法，主要通过根据进程的等待时间和运行时间来调整其优先数。

具体来说，等待时间越长，运行时间越短，优先数就会相应增加，从而使该进程在队列中的优先级提高。

概念区别

优先权调度算法分为两种，分别是**抢占式**和**非抢占式**。

优先权分为两种，分别是**静态**优先权和**动态**优先权

四、时间片轮转调度算法 RR

分时系统中广泛使用，时间片大小：10~100 ms

含义：系统将所有就绪进程按先来先服务的原则，排成一个队列，**每次调度时把CPU分给队首进程，并令其执行一个时间片。**

当时间片用完时，调度程序终止当前进程的执行，并将它送到就绪队列的队尾。

时间片大小的确定：

系统对响应时间的要求：响应时间要求越短，时间片越小

就绪队列中进程的数目：进程数量越多，时间片越小

系统的处理能力：处理能力越好，时间片越小

性能评价：

时间片很大 = 先来先服务算法

时间片很小：增大cpu对进程切换和调度的开销

五、多级队列调度算法

含义：将就绪队列分成多个独立队列，每个队列有自己的调度算法。

解释说明：

队列之间可采取固定优先级、时间片划分。如：

① 固定优先级：高优先级空时低优先级进程才能被调度；

② 时间片划分：如三个队列分配时间50%、40%、10%。

队列内调度算法，如：

① 系统进程队列采用优先级调度；

② 交互式队列采用时间片轮转调度算法 RR；

③ 批处理队列采用先来先服务调度算法 FCFS

六、多级反馈队列调度算法

含义：建立多个优先权不同的就绪队列，每个队列有大小不同的时间片。

解释说明：

队列之间可采取固定优先级，如：

固定优先级：高优先级空时低优先级进程才能被调度；

各队列可采用不同的调度策略，如：

① 新进程到达时先进入第1级队列，按FCFS调度；

② 若用完时间片进程还未结束，则进程进入下一级队列队尾。

新进程加入最高优先级队列，时间片用完则降级到次优先级队列中。

3.3 实时系统中的调度

实现实时调度的基本条件

1) 提供必要的调度信息：

就绪时间、开始截止时间、完成截止时间

口诀：初九开完会的时间就自由了

处理时间、资源要求、优先级

- 2) 系统处理能力强
- 3) 采用抢占式调度机制
- 4) 具有快速切换机制

常用的实时调度算法

最早截止时间优先算法 EDF

开始截止时间越早，进程优先级越高，越优先获得CPU

最低松弛度优先算法LLF

进程的紧迫程度: $L = T - T_c - T_s$ T: 完成截止时间;

T_c : 当前时间;

T_s : 处理完该任务还需要的时间

例子:

淘宝: $L_1 = 12:40 - 12:10 - 20 = 10$ 分钟

京东: $L_2 = 12:30 - 12:10 - 01 = 19$ 分钟

$L_1(\text{淘宝}) < L_2(\text{京东})$, $L_1(\text{淘宝})$ 优先。

3.4 进程切换

进程切换

进程切换的含义

当前正在执行的进程成为被替换进程，让出其所使用的CPU，以运行被进程调度程序选中的新进程。

进程切换的步骤

	1) 保存CPU上下文环境 2) 更新进程1控制块 3) 修改进程1状态 (执行——>就绪或阻塞) 4) 移动进程控制块到就绪队列或阻塞队列 5) 调度新程序, 更新进程2控制块 6) 更新内存管理的数据结构 7) 恢复进程2的硬件上下文
1) 保存包括程序计数器和其他寄存器在内的CPU上下文环境 2) 更新被替换进程的进程控制块 3) 修改进程状态, 把执行态改为就绪态或阻塞态 4) 将被替换进程的进程控制块移到就绪队列或阻塞队列 5) 执行通过进程调度程序选择的新进程, 并更新该进程的进程控制块 6) 更新内存管理的数据结构 7) 恢复被调度程序选中的进程的硬件上下文	

3.5 多处理器调度

多处理器系统的类型:

多处理器系统:

根据耦合程度分: 紧密耦合: 共享主存储器和I/O设备; 松弛耦合: 有各自的存储器和I/O设备	根据结构功能分: 对称: 处理单元功能和结构相同; 非对称: 有多种类型的处理单元; 一个主处理器, 多个从处理器
---	--

多处理器系统的进程分配方式

对称系统分配方式

静态分配: 就绪队列的进程只能在与就绪队列对应的处理器上运行

动态分配: 进程随机地被分配到当时处于空闲状态的某一处理器上执行

非对称系统分配方式

主-从式操作系统: 主机执行调度程序, 从机运行程序; 从机请求进程, 主机分配进程。

进程 (线程) 调度方式

自调度：

最常用最简单的方式

任何一个空闲处理器都可以从就绪队列中选取一个进程或线程运行（先来先服务调度算法）

优点：易移植，很容易将单处理器环境下所用的调度机制移植到多处理器环境中；

有利于提高CPU的利用率，不会出现处理器空闲或忙闲不均的情况。

缺点：瓶颈问题，处理器数量很多时；

低效性，多次更换处理器；

线程切换频繁，某些线程因其合作的线程未获得处理器而阻塞导致进程切换。

成组调度：

系统将一组相互合作的进程或线程同时分配到一组处理器上运行，进程或线程与处理器一一对应。

优点：减少线程切换；

减少调度开销。

专用处理器分配：

在程序执行期间，专门为该程序分配一组处理器，每个线程一个

优点：加快程序运行速度；

避免进程切换

缺点：处理器资源严重浪费

3.6 死锁

死锁的定义

死锁：由于多个进程竞争共享资源而引起的进程不能向前推进的僵死状态

产生死锁的原因和必要条件

产生原因

竞争共享资源且分配资源的顺序不当

必要条件（只有当四个条件同时满足时才会产生死锁）

- 1) 互斥条件
- 2) 请求和保持条件
- 3) 不剥夺条件
- 4) 环路等待条件

处理死锁的基本方法

死锁的预防

通过破坏死锁的产生条件来保证不发生死锁

保证至少其中一个产生死锁的条件不成立

- 1) 摒弃请求和保持条件
- 2) 摒弃不剥夺条件，缺点：实现复杂，代价高
- 3) 摒弃环路等待条件，进程必须按规定的顺序申请资源

缺点：限制了新设备的增加，资源浪费，用户编程麻烦

死锁的避免

通过算法合理分配资源来保证不发生死锁

通过资源合理分配使系统处于安全状态

能够找到一个进程执行序列，按照这个序列为每个进程分配资源，就可以保证进程资源分配和执行顺利完成，不会发生死锁

- 1) 安全状态：至少找到一个这样的序列，不可能发生死锁
- 2) 不安全状态：找不到一个这样的序列，可能会发生死锁

死锁避免算法（银行家算法 -- Dijkstra）

一个进程提出资源请求后，系统先进行资源的试分配，分配后检测系统是否安全。

进程名称	allocation已分配 (A B C)	Max最大需求 (A B C)	Need还需要 (A B C)	Available可用 (A B C)
p0	0 1 0	7 5 3	7 4 3	3 3 2
p1	2 0 0	3 2 2	1 2 2	
p2	3 0 2	9 0 2	6 0 0	
p3	2 1 1	2 2 2	0 1 1	
p4	0 0 2	4 3 3	4 3 1	

对work和finish初始化

`work=available=(3,3,2)`

`finish[0]=finish[1]=finish[2]=finish[3]=finish[4]=false`

循环检测是否能找到一个进程 p_i ，满足 `finish[i]=false` 且 `needi≤work`

因为 `finish[1]=false` 且 `need1<work`

所以 `finish[1]=true`, `work=work+allocation1=(3,3,2)+(2,0,0)=(5,3,2)`

...

到此，再也找不到满足 `finish[i]=false` 且 `needi≤work` 的进程，循环结束

所有进程 `finish[i]=ture`，安全

死锁的检测

检测当前系统是否出现死锁

何时调用检测算法：

死锁可能发生的频率，

死锁发生时受影响的进程数量

资源分配图：

指向进程→分配给进程；

进程发出→进程请求

死锁定理：

用于检测系统所处的资源分配状态是否为死锁状态

S为死锁状态的充分条件是当且仅当S状态的资源分配图是不可完全简化的

死锁的解除

检测到系统有死锁后进行解除，

解除途径：

1) 进程终止：

① 终止所有死锁进程，

② 一次只终止一个处于死锁的进程，直到死锁解除

2) 资源抢占：逐步从进程中抢占资源给其他进程使用直到死锁被打破为止

① 抢占的代价最小，

死锁进程拥有的资源数量、死锁进程到目前为止已经消耗的执行时间

② 回滚，回到程序当初的状态

③ 饥饿，一直被剥夺资源，得不到运行。